

Módulo 4: Machine Learning

Día 07: Modelos de Ensamble (La Sabiduría Colectiva)

Diplomado en Ciencia de Datos

Semana 3

¿Por qué fallan los modelos individuales?

Hasta ahora, hemos confiado nuestras predicciones a un solo ".experto" (un Árbol de Decisión o una Regresión Logística).

El Problema de los Árboles Individuales:

- **Alta Varianza (Overfitting):** Si dejamos que un árbol crezca sin control, memorizará todos los detalles del dataset (incluyendo el ruido).
- Si le mostramos datos ligeramente nuevos, fallará catastróficamente.

Solución: Ensamble (La Sabiduría de las Multitudes) En lugar de preguntarle a un solo experto brillante pero temperamental, le preguntaremos a 100 expertos promedio y promediaremos sus respuestas.

Concepto 2: Boosting

Objetivo: Reducir el Sesgo (el Underfitting).

Algoritmos Estrellas: *Gradient Boosting, XGBoost.*

¿Cómo funciona?

- 1 Entrenamos un modelo muy simple (débil) que cometerá muchos errores.
- 2 Calculamos exactamente en qué registros se equivocó.
- 3 Entrenamos un **segundo modelo** que se especialice EXCLUSIVAMENTE en corregir los errores del primer modelo.
- 4 Repetimos secuencialmente cientos de veces.

Boosting (Ej: XGBoost) - Trabajo Secuencial



¿Por qué la industria financiera y el retail aman los Ensamblados?

- **Fraude Bancario:** XGBoost es el rey de las competencias de Kaggle porque es extremadamente preciso detectando anomalías sutiles en secuencias de transacciones.
- **Fuga de Clientes (Churn):** Random Forest proporciona una excelente "Importancia de Variables" (Feature Importance), permitiendo al negocio saber *por qué* se van los clientes, no solo *quiénes* se van.

Hiperparámetros Críticos: Random Forest

Para evitar que nuestro “Consejo de Sabios” se salga de control, debemos limitar su crecimiento:

- `n_estimators`: Número de árboles en el bosque (ej. 100 o 500). Más árboles = más estabilidad, pero mayor costo computacional.
- `max_depth`: Profundidad máxima de cada árbol. Evita que los árboles memoricen a clientes individuales.
- `class_weight='balanced'`: **¡Vital en Fraudes!** Obliga al bosque a prestarle la misma atención a la clase minoritaria (fraude) que a la mayoritaria (legítimo).

Hiperparámetros Críticos: XGBoost

El modelo evolutivo es más poderoso, pero requiere más "sintonización" matemática:

- `learning_rate` (Tasa de aprendizaje): ¿Qué tan agresivo es el modelo al corregir los errores del árbol anterior? Valores pequeños (0.01) requieren más árboles, pero generalizan mejor.
- `n_estimators`: Cuántas iteraciones de corrección de errores permitiremos.
- `subsample`: Qué porcentaje de datos usamos en cada iteración para evitar el sobreajuste.

El Poder de la "Importancia de Variables"

Problema: La junta directiva no entiende qué es un Bosque Aleatorio. Solo quieren saber por qué se van los clientes.

Feature Importance (Importancia de Atributos):

- Los modelos de Ensamble pueden calcular matemáticamente cuántas veces una variable (ej. *Retraso en Pagos*) fue utilizada para tomar una decisión crítica.
- Nos devuelve un ranking de impacto.
- **Impacto:** Permite pasar de ser un modelo "predictivo" a ser una herramienta de "diagnóstico" para el negocio.

Ventajas:

- **Rendimiento superior:** Rara vez son superados en datos tabulares (Excel/SQL).
- **Estabilidad:** Inmunes a valores atípicos severos.
- **Versátiles:** Manejan relaciones no lineales sin esfuerzo.

Desventajas:

- **Caja Negra:** Es imposible dibujar o imprimir un modelo compuesto por 500 árboles. Se pierde la interpretabilidad visual directa.
- **Costo Computacional:** Entrenar cientos de modelos toma significativamente más tiempo y memoria RAM.

¿Cómo toma decisiones matemáticamente un Ensamble?

1. En Clasificación (Ej. Fraude o No Fraude):

- **Random Forest:** Utiliza "Voto Mayoritario Suave" (*Soft Voting*). Promedia las probabilidades matemáticas estimadas por sus 100 árboles. Si el promedio de todos los árboles indica un 65 % de probabilidad de Fraude, la predicción final es Fraude.
- **XGBoost:** Asigna un "peso.^a cada árbol. Los árboles secuenciales que lograron reducir la Tasa de Error (*Loss Function*) más drásticamente tienen un voto que vale el doble o el triple al final.

2. En Regresión (Ej. Predecir Ventas en Dólares):

- Ambos algoritmos calculan el promedio aritmético exacto de las estimaciones numéricas de todos los árboles del ensamble.

Comandos Útiles en Python

Implementar estos modelos requiere solo unas líneas de código:

```
# 1. Random Forest (Bagging)
from sklearn.ensemble import RandomForestClassifier

bosque = RandomForestClassifier(n_estimators=100, class_weight='balanced')
bosque.fit(X_train, y_train)
pred_rf = bosque.predict(X_test)
importancia = bosque.feature_importances_ # <-- Auditor a de variables

# 2. XGBoost (Boosting Evolutivo)
from xgboost import XGBClassifier

modelo_xgb = XGBClassifier(learning_rate=0.1, n_estimators=100)
modelo_xgb.fit(X_train, y_train)
pred_xgb = modelo_xgb.predict(X_test)
```

¡Vamos a la práctica!

Aplicaremos estos conceptos en Python para detectar Fuga de Clientes y Fraude Financiero.